

SSN College of Engineering, Kalavakkam
Department of Computer Science and Engineering
V Semester - CSE 'A'
UCS2511 NETWORKS LAB
Exercise 11
Error Control

Name : Aathish Sivasubrahmanian 3122215001001

Aim:

Simulation of Error Correction Mechanism via Hamming Code in C

O/P:

Program

1. Server Side

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <fcntl.h>
#include <stdbool.h>
#include <math.h>
#define MAXLINE 1024
#include "Hamming.h"

int main(int argc, char ** argv)
{
    2
    srand(time(NULL));
    if (argc < 2){
        fprintf(stderr, "Enter port number as argument!\n");
        exit(EXIT_FAILURE);
    }
```

```

int PORT = atoi(argv[1]);
int sockfd, newfd, n, arr[30], count = 0, bin;
char buff[MAXLINE], buffer[MAXLINE], data_t[40];
int i, j, r, total, nob, rem, dig, pos;
long data;
struct sockaddr_in servaddr, cliaddr;
if((sockfd = socket(AF_INET, SOCK_STREAM, 0)) < 0)
{
    perror("Socket creation failed!");
    exit(1); }
bzero(&servaddr, sizeof(servaddr)); servaddr.sin_family = AF_INET; // IPv4
servaddr.sin_addr.s_addr = INADDR_ANY; servaddr.sin_port = htons(PORT);
if(bind(sockfd, (const struct sockaddr *)&servaddr, sizeof(servaddr)) < 0)
{
    perror("Bind failed!");
    exit(1); }
int len, m;
listen(sockfd, 2);
printf("Enter the data to send: ");
scanf("%lu", &data);
n = countbits(data);
r = log2(n);
r = floor(r);
// Finding number of redundant bits
while(pow(2, r) < n + r + 1)
    r += 1;
printf("\nNo. of redundant bits required: %d\n", r);
total = n + r;
nob = floor(log2(total));
for(i = 1; i <= total; i++)
{
    dig = data % 10;
    if(isapower2(i) == 0)
    {
        arr[total - i] = dig;
        data /= 10; }
    else
        arr[total - i] = 0;
    for(i = 0; i < r; i++)
    }
}
3

{
    for(j = 1; j <= total; j++)
    {
        if((int)(pow(2, i)) != j)
        {

```

```

}}
}
2. Client Side
#include <stdio.h> #include <sys/types.h> #include <sys/socket.h> #include
<netinet/in.h> #include <string.h> #include <unistd.h> #include <arpa/inet.h>
#include <stdlib.h> #include <math.h> #define MAXLINE 1024
bin = binary(j);
if(ispresent(bin, i + 1))
    count += arr[total - j];
    if(count % 2 == 0)
        arr[total - (int)(pow(2, i))] = 0;
else
    arr[total - (int)(pow(2, i))] = 1;
    count = 0;
}
printf("\nData with redundant bits: ");
for(i = 0; i < total; i++)
    printf("%d", arr[i]);
// printf("\nEnter error position: "); // scanf("%d", &pos);
pos = rand() % total + 1;
printf("\nIntroducing error automatically at bit: %d\n", pos);
if(arr[total - pos] == 0)
    arr[total - pos] = 1;
else
    arr[total - pos] = 0;
int k = 0;
long num = 0;
for(i = total - 1; i >= 0; i--)
{
    num += pow(10, k) * arr[i];
    k++; }
sprintf(data_t, "%lu", num);
printf("Data transmitted is %s\n", data_t);
len = sizeof(cliaddr);
newfd = accept(sockfd, (struct sockaddr*)&cliaddr, &len);
m = write(newfd, data_t, sizeof(data_t));
#include "Hamming.h"
4

int main(int argc, char **argv)
{
    if (argc < 2){
        fprintf(stderr, "Please enter port number as second argument!\n");
        exit(EXIT_FAILURE);
    }
    int PORT = atoi(argv[1]);
    long num;

```

```

    int sockfd, total, i, rem, arr[20], count = 0, r = 0, result = 0, bin, j,
    ,→ newarr[20], finalarr[20];
    char buffer1[40];
    struct sockaddr_in servaddr;
    if ((sockfd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("Socket creation failed!");
        exit(1); }
    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(PORT);
    servaddr.sin_addr.s_addr = inet_addr("127.0.0.1");
    int n, len;
    connect(sockfd, (struct sockaddr*)&servaddr, sizeof(servaddr));
    n = read(sockfd, buffer1, sizeof(buffer1));
    num = atol(buffer1);
    total = countbits(num);
    printf("Received data: %lu\n", num);
    i = 1;
    while(num > 0)
    {
        rem = num % 10;
        arr[total - i] = rem;
        num /= 10;
        i++;
    }
    for(i = 1; i <= total; i++)
        if(ceil(log2(i)) == floor(log2(i)))
            r++;
    int k = 0;
    for(i = 0; i < 4; i++)
    {
        for(j = 1; j <= total; j++)
        {
            bin = binary(j);
            if(ispresent(bin, i + 1))
                count += arr[total - j];
        }
        if(count % 2 == 0)
            result += pow(10, k) * 0;
        else
            result += pow(10, k) * 1;
        k++;
    }
    count=0; }
    int error = decimal(result);
    printf("\nError bit in binary: %d\n", result);
    printf("\nError in bit %d\n", error);
    if(arr[total - error] == 0)

```

```

arr[total - error] = 1;
5

else
    arr[total - error] = 0;
    k = 0;
    printf("\nData after error correction: ");
    for(i = total - 1; i >= 0; i--)
    {
        newarr[k] = arr[i];
k++; }
    int x = 0;
    for(i = 0; i < k; i++)
    {
        if(ceil(log2(i + 1)) != floor(log2(i + 1)))
        {
            finalarr[x] = newarr[i];
x++; }
    }
    for(i = x - 1; i >= 0; i--)
        printf("%d", finalarr[i]);
    printf("\n");
return 0; }

```

3. Hamming Code specific functions

```

int countbits(long num)
{
    int r, count = 0;
    while(num > 0)
    {
num = num / 10;
count++; }
    return count;
}

int binary(int num)
{
    int bin = 0, r;
    int i = 0;
    while(num > 0)
    {
        r = num % 2;
        bin += r * pow(10, i);
        num /= 2;
        i++;
    }
    return bin; }

int ispresent(int num, int pos)
{

```

```

    int rem;
    for(int i = 0; i < pos; i++)
    {
rem = num % 10;
        num = num / 10;
    }
    if(rem == 1)
        return 1;
6
else
    return 0;
}
int isapower2(int n)
{
    if(ceil(log2(n)) == floor(log2(n)))
        return 1;
    else
        return 0;
}
int decimal(int num)
{
    int rem, i = 0, result;
    while(num > 0)
    {
        rem = num % 10;
        result += pow(2, i) * rem;
        num /= 10;
        i++;
    }
    return result;
}

```

Output:

```
→ ./server 8080
Enter the data to send: 1010101

No.of redundant bits required: 4

Data with redundant bits: 10100101111
Introducing error automatically at bit: 5
Data transmitted is 10100111111
```

```
→ ./client 8080
Received data: 10100111111

Error bit in binary: 101

Error in bit 5

Data after error correction: 1010101
```

Learning Outcomes:

I learned how to implement Hamming Error correction using socket programming.